



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



LAB MANUAL

Python Email Automation & SMTP

Two-Lecture Practical Manual

Based on the lecture presentations: "Introduction to Email Automation & SMTP | Python Basics"



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



1. Lab Manual Overview

This manual converts the two lecture presentations into a practical, student-facing learning sequence. The first lecture establishes the networking foundation required for email automation: what SMTP is, what an SMTP server does, why ports and encryption matter, how Python uses the `smtplib` module, and how a secure authenticated connection is created. The first laboratory task deliberately stops after successful authentication so that students understand the connection lifecycle before sending a message. The second lecture extends that foundation by constructing the actual email message with `EmailMessage`, adding headers and body content, and transmitting the completed message through the already-understood SMTP connection.

The central idea throughout the manual is that email automation contains two different responsibilities. The email package constructs the message data, while `smtplib` handles the network communication with the SMTP server. Keeping those responsibilities separate makes the program easier to understand and gives students a clear mental model of the complete workflow.

Source basis: The explanations, terminology, workflow, code examples, safety constraints, mini task, and future-pathway topics in this manual are grounded in the uploaded lecture presentation. The source explicitly defines the lecture objectives, SMTP concepts, secure connection flow, `EmailMessage` construction, `send_message` workflow, error cases, and student email-sender project.

2. Learning Objectives

By the end of the two lectures and associated practical work, students should be able to explain the purpose of SMTP in an email automation workflow, distinguish SMTP from email retrieval protocols at a basic level, identify the role of an SMTP server and network port, establish a secure SMTP connection



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



with Python, authenticate with a server, construct a plain-text email using EmailMessage, and transmit the message using send_message().

Students should also be able to recognize common beginner failures such as incorrect email addresses, invalid app-password credentials, SMTP port blocking, and general network connectivity problems. The second practical activity combines these concepts into a small interactive Student Email Sender program.

3. Lecture 1 — Core SMTP Connections

Lecture 1 focuses on the infrastructure that makes email automation possible. At this stage, the objective is not to build a sophisticated email application. The objective is to understand how a Python program reaches a mail server securely and proves its identity. The practical endpoint is a successful authenticated SMTP connection; students do not send a message during the Lecture 1 verification task.

3.1 What Is Email Automation?

Email automation means allowing a software program to perform email communication without requiring a person to manually compose and send every message. In the lecture presentation, automation is presented as a way to replace repetitive manual communication with a programmatic workflow. The major advantage is that a software system can trigger communication at the appropriate time or in response to a system event. This is particularly useful when the same communication pattern occurs repeatedly.

The presentation uses several real-world applications to make this idea concrete. Attendance notifications can be triggered when a database record changes. Registration confirmations can be sent immediately after a new user signs up. Password-reset messages can be delivered when a user requests a reset. Weekly reports can be scheduled automatically, while critical downtime alerts can be broadcast when a service problem is detected. These examples show why email is a useful target for automation: the information already exists inside a software system, and a Python program can turn that information into an outgoing communication.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Key idea: Automation does not begin with the email body.

It begins with a system event or a piece of information that the program needs to communicate. SMTP is the communication mechanism used to move the outgoing message toward its destination.

3.2 How an Email Travels

The presentation describes email delivery as a network of clients and servers. A sender uses an email client or application to create a message. In our case, the client can be a custom Python script. That client connects to an outgoing mail server and hands the message to that server. The outgoing server then routes the message across the Internet, while the recipient ultimately accesses the delivered message through an email client.

For the automation exercises, the presentation simplifies this process so students can concentrate on the part controlled directly by Python. The Python script generates the message, passes it to an SMTP server, and the SMTP server handles the outgoing delivery path. Thinking about the workflow this way prevents a common beginner misunderstanding: the Python script is not itself the Internet-wide email delivery infrastructure. It communicates with the infrastructure that performs that job.

Simplified workflow: Python script → SMTP server → Internet → recipient inbox. The presentation identifies this as the simplified flow used for the automation projects.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



3.3 SMTP — Simple Mail Transfer Protocol

SMTP stands for Simple Mail Transfer Protocol. The presentation defines SMTP as the foundational Internet standard used for email transmission across IP networks and emphasizes that its primary purpose is sending or routing outgoing email traffic. In practical terms, SMTP is the protocol that allows an email client or program to communicate with an outgoing mail server and allows mail servers to relay messages onward.

It is useful to distinguish SMTP from the protocols that retrieve messages from an inbox. The presentation mentions IMAP and POP3 as examples of protocols used for retrieving or downloading mail. Therefore, when students see `smtp` in a Python program, they should immediately associate it with the outgoing or sending side of email communication rather than with reading an inbox.

```
import smtplib
```

This import makes Python's standard-library SMTP client functionality available to the program. The module provides the objects and methods needed to establish an SMTP session and communicate with an SMTP server. Students do not need to install `smtp` separately because it is included with Python's standard library.

3.4 What Is an SMTP Server?

An SMTP server is the dedicated email infrastructure responsible for receiving, routing, and delivering outgoing messages. A local Python program cannot simply place a message directly into a remote recipient's inbox. Instead, it hands the message to an SMTP server that is designed to perform outgoing email routing. This is why an SMTP server is a required part of the Python email-automation workflow.

SMTP servers are provider-specific. The presentation gives Gmail as the working example and identifies `smtp.gmail.com` as the SMTP server address. When a program connects, it must target the correct outgoing server for the sender's email provider and use the corresponding configuration.

```
smtp.gmail.com
```



3.5 Network Ports

A network port is a virtual endpoint that helps direct network traffic to the correct service on a server. The presentation explains that ports are needed so SMTP traffic can be distinguished from other network services such as web traffic or file transfer. In the Gmail examples, port 465 is used for SMTP over SSL, while port 587 is identified as the modern port for SMTP with STARTTLS.

For the beginner implementation in these lectures, port 465 is used together with `SMTP_SSL()`. This keeps the code and the security model straightforward: the connection is created as an encrypted SMTP session from the start. Port 587 is introduced conceptually so students recognize the alternative, but it is not used in the core practical code.

Important distinction: A port number is not the SMTP protocol itself. SMTP describes the communication protocol, while the port identifies the network endpoint used to reach the corresponding service.

3.6 SSL/TLS and Secure Connections

Email automation involves sensitive information because authentication credentials must be sent to the mail server. An unencrypted connection could expose those credentials to network interception. The presentation therefore introduces SSL and TLS as cryptographic technologies used to secure network communication. It explains that `SMTP_SSL()` creates an encrypted connection from the beginning of the session.

The presentation pairs `smtp.SMTP_SSL()` with `ssl.create_default_context()`. The SSL context provides trusted system certificates and is used as part of the secure connection setup. For students, the



important conceptual point is that encryption is not an optional decoration added after the login step; it protects the connection used by the login and subsequent SMTP communication.

```
import ssl  
  
context = ssl.create_default_context()
```

3.7 The smtplib Module

A Python module is a reusable collection of definitions and statements. smtplib is part of the Python standard library and provides an SMTP client session object. The presentation describes it as the networking component of the automation solution. It can establish connections to SMTP servers, support authentication, and transmit data without requiring students to work directly with the low-level socket operations underneath the communication.

At the beginner level, the most important mindset is to treat smtplib as the bridge between the Python application and the email provider's outgoing mail infrastructure. The application decides what it wants to send; smtplib provides the network mechanism used to hand that information to the SMTP server.

3.8 Establishing the Secure SMTP Connection

The fundamental connection statement in the presentation is shown below. It creates an SMTP_SSL object using the Gmail SMTP server address and port 465.

```
server = smtplib.SMTP_SSL("smtp.gmail.com", 465)
```

Each element has a precise role. smtplib refers to the imported Python module. SMTP_SSL is the class used to establish an encrypted SMTP session. The string smtp.gmail.com identifies the outgoing server. The integer 465 identifies the secure SMTP endpoint. Together, these values establish the communication path between the local Python script and the remote mail server.

Mental model: Think of the connection line as opening a secure communication pipeline. At this moment, the program has



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



not yet proven the sender's identity and has not sent an email.
only established the network session.

It has

3.9 SMTP Authentication

Once the secure network session is established, the program must authenticate. Authentication tells the SMTP server which account is attempting to send mail and allows the server to verify that identity before accepting outgoing messages. The presentation shows authentication through the `server.login()` method.

```
server.login(sender_email, password)
```

The sender email is used as the account identity, while the password argument represents the credential required for authentication. The lecture presentation strongly emphasizes safe credentials. It recommends using a dedicated testing account with two-factor authentication and an app password rather than exposing primary personal credentials in plain text code. The purpose of this practice is to isolate the classroom automation activity from a primary personal mailbox and reduce the risk associated with credential exposure.

Security rule for the lab: Never paste a primary personal email password into a classroom source file. Follow the credential approach specified by the lecture: use a designated testing account and an app password for the SMTP exercise.



3.10 Lecture 1 Secure Connection Program

The complete Lecture 1 demonstration combines smtplib, ssl, a secure SSL context, the Gmail SMTP endpoint, and the login operation. Notice that the code does not construct or transmit a message. Its only goal is to demonstrate the connection lifecycle.

```
import smtplib
import ssl

context = ssl.create_default_context()

with smtplib.SMTP_SSL("smtp.gmail.com", 465, context=context) as server:
    server.login(sender_email, password)
    print("Connected successfully!")
```

The with statement is important because it manages the connection lifecycle. When the block finishes, the SMTP connection is closed automatically. This is cleaner than manually keeping the connection open and remembering to close it in every possible execution path. The presentation identifies the lifecycle as Connect, Authenticate, and Close.

3.11 Lecture 1 Lab Task — SMTP Verification

The first practical task is deliberately narrow. Students create a small verification script that checks whether their environment, SMTP server settings, secure connection, and authentication credentials are working. The task ends after successful authentication; it does not send an email.

```
import smtplib
import ssl

context = ssl.create_default_context()

with smtplib.SMTP_SSL("smtp.gmail.com", 465, context=context) as server:
    server.login(sender_email, password)
    print("SMTP Login Successful")
```

Save the program as smtp_verify.py and run it from the terminal:

```
python smtp_verify.py
```

The expected successful result is:

```
SMTP Login Successful
```



If the program reaches this output, the student has successfully completed the core Lecture 1 milestone: the Python application can establish a secure SMTP session and authenticate with the configured account. The presentation explicitly instructs students not to send an email in this task because the teaching objective is to master the connection and authentication mechanics first.

4. Lecture 2 — Constructing and Sending Email

Lecture 2 begins from the point where Lecture 1 stops. The connection and authentication foundation is already understood. The new problem is message construction: before the SMTP server can transmit an email, Python must have an actual email message object containing routing information and body content. The presentation makes this transition explicit by adding a new stage, Create Email, between Authenticate and Send Email.

4.1 The Complete Email Automation Lifecycle

The full workflow taught across the two lectures can be understood as a sequence of five stages. The program starts by executing Python code, creates the email message, opens a secure SMTP connection, authenticates the sender, sends the constructed message, and then closes the connection. Keeping these stages conceptually separate makes debugging much easier because a failure can be associated with a particular part of the workflow.

```
Python Script
↓
Create Email
↓
Connect to SMTP Server
↓
Authenticate
↓
Send Email
↓
Close Connection
```

4.2 Anatomy of an Email



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



A standard email contains structured headers and a main body. The presentation introduces three essential routing headers for this beginner exercise: From identifies the sender, To identifies the recipient, and Subject provides a short summary or title. The body contains the actual message content that the recipient reads.

```
From: teacher@gmail.com
To: student@gmail.com
Subject: Python Assignment

Hello Student,
Your assignment has been received.
```

The separation between headers and body matters when an email is constructed programmatically. Headers describe and route the message, while the body contains the human-readable payload. EmailMessage gives Python a structured way to represent both parts without requiring the student to manually format the low-level Internet message syntax.

4.3 The email Package and EmailMessage

The second lecture introduces EmailMessage with the following import statement:

```
from email.message import EmailMessage
```

The presentation uses a clear division of responsibilities. The email package is the data layer: it creates and structures the actual message. smtplib is the network layer: it communicates with the SMTP server, performs authentication, and handles delivery of the constructed message. This distinction is one of the most important conceptual outcomes of the lesson.

Remember: email.EmailMessage creates the message. smtplib sends the message through an SMTP server. Students should not think of these two libraries as interchangeable; they solve different parts of the problem.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



4.4 Creating an EmailMessage Object

EmailMessage is a Python class designed to represent an email in memory. Creating an instance of the class gives the program an empty, structured container into which the routing headers and body can be inserted.

```
msg = EmailMessage()
```

The variable msg now refers to the message object. At this point it does not contain the sender, recipient, subject, or body. Those elements are added in the next steps.

4.5 Setting the Email Headers

The presentation uses dictionary-style syntax to assign the primary email headers. The left side of each assignment identifies the header name, while the right side provides the associated value. This syntax hides the complex low-level formatting that would otherwise be required when constructing a raw Internet email message.

```
msg["From"] = sender_email  
msg["To"] = receiver_email  
msg["Subject"] = "My First Python Email"
```

The From field identifies the sender address, the To field identifies the destination address, and the Subject field provides the title displayed by the recipient's email client. The values can be fixed strings, or they can come from variables. Using variables is especially useful for automation because the same program can be reused with different recipients and different message content.

4.6 Setting the Email Body

The message body is added with the set_content() method. The lecture uses plain text as the first implementation because it keeps the body simple and avoids introducing HTML or additional message formatting concepts.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
msg.set_content(  
    "Hello! This email was sent using Python."  
)
```

Here, msg is the EmailMessage object and set_content() is the method used to inject the body text. The string passed to the method becomes the plain-text content of the email. The presentation emphasizes that dictionary-style syntax is used for metadata headers, while set_content() is used for the actual body.

4.7 Sending the Message with send_message()

Once the EmailMessage object is complete, it can be passed to smtplib through the send_message() method. This is the command that connects the message-construction phase to the network-transmission phase.

```
server.send_message(msg)
```

The message object contains the headers and body that were created earlier. The SMTP server connection, which has already been authenticated, accepts that object and transmits it as an outgoing email. The presentation summarizes the final workflow as Create, Connect, Login, Send, and Close.

4.8 Complete Email Automation Program

The following script combines the full workflow taught in the two lectures. It imports the required libraries, creates the message, populates the headers and body, creates a secure SSL context, connects to the SMTP server, authenticates, and finally sends the message.

```
import smtplib  
import ssl  
from email.message import EmailMessage  
  
sender_email = "sender@gmail.com"  
receiver_email = "receiver@gmail.com"  
  
msg = EmailMessage()  
msg["From"] = sender_email  
msg["To"] = receiver_email  
msg["Subject"] = "Python Test Email"  
msg.set_content("Hello! This email was sent using Python.")  
  
context = ssl.create_default_context()  
with smtplib.SMTP_SSL(  

```



```
"smtp.gmail.com",  
465,  
context=context  
) as server:  
server.login(sender_email, password)  
server.send_message(msg)  
  
print("Email sent successfully!")
```

A student should be able to explain this program in four major stages. First, the libraries provide the networking, security, and message-construction functionality. Second, the EmailMessage object is created and populated. Third, the SSL context is created and the authenticated SMTP connection is opened. Fourth, send_message() transmits the finished message, after which the with block automatically closes the connection.

4.9 Basic Error Handling with try/except

Network automation can fail for reasons outside the Python syntax itself. The presentation therefore introduces try/except as a way to catch an error and report it instead of allowing the whole script to terminate without a useful explanation. In a beginner program, this can be as simple as wrapping the email operation in a try block and printing the exception in the except block.

```
try:  
    # email execution code  
    print("Email sent!")  
except Exception as e:  
    print("Error:", e)
```

The presentation identifies several common failure categories. Email-address mistakes can prevent a valid message from being created or delivered. Invalid app-password credentials can cause authentication to fail. A firewall, ISP, or network policy can interfere with the SMTP port. General Internet connectivity problems can prevent the SMTP handshake from completing. When troubleshooting, students should identify which stage failed rather than changing the entire program at once.

4.10 Mini Project — Student Email Sender

The final practical activity combines the concepts from both lectures into a small interactive application. The goal is to replace fixed recipient, subject, and message values with information collected from the



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



user at runtime. This demonstrates the difference between a one-off demonstration script and a reusable automation program.

The program should ask the user for a receiver email address, a subject line, and the message body. These values should be stored in variables and then assigned to an EmailMessage object. The program should establish the secure SMTP connection using smtplib, authenticate with the configured account, transmit the message with send_message(), and protect the execution with try/except so that common failures are reported clearly.

```
import smtplib
import ssl
from email.message import EmailMessage

sender_email = "sender@gmail.com"
receiver_email = input("Enter receiver email: ")
subject = input("Enter subject: ")
message_body = input("Enter message: ")

password = "YOUR_APP_PASSWORD"

msg = EmailMessage()
msg["From"] = sender_email
msg["To"] = receiver_email
msg["Subject"] = subject
msg.set_content(message_body)

try:
    context = ssl.create_default_context()

    with smtplib.SMTP_SSL(
        "smtp.gmail.com",
        465,
        context=context
    ) as server:
        server.login(sender_email, password)
        server.send_message(msg)

    print("Email sent successfully!")

except Exception as e:
    print("Error:", e)
```

The three input() calls create the dynamic part of the application. Instead of changing the source code every time a different person needs an email, the user can supply the destination and content when the program runs. This is the basic pattern that can later be extended into automated notifications, reports, and system alerts.



4.11 Code Walkthrough — Line by Line

The import section loads the three required capabilities. `smtplib` supplies the SMTP client functionality. `ssl` provides the security context. `EmailMessage` provides the structured email message object.

```
import smtplib
import ssl
from email.message import EmailMessage
```

The configuration section stores sender and receiver information and, in the interactive project, collects the recipient, subject, and message dynamically. These values are ordinary Python variables, so students can revise them just as they would in any other Python program.

```
sender_email = "sender@gmail.com"
receiver_email = input("Enter receiver email: ")
subject = input("Enter subject: ")
message_body = input("Enter message: ")
```

The message construction section creates the `EmailMessage` object, adds its routing headers, and places the body text inside the object.

```
msg = EmailMessage()
msg["From"] = sender_email
msg["To"] = receiver_email
msg["Subject"] = subject
msg.set_content(message_body)
```

The networking section creates a secure SSL context, opens the Gmail SMTP connection on port 465, authenticates, and sends the message. Because the code is inside a `with` statement, the connection is automatically closed when the block finishes.

```
context = ssl.create_default_context()

with smtplib.SMTP_SSL(
    "smtp.gmail.com",
    465,
    context=context
) as server:
    server.login(sender_email, password)
    server.send_message(msg)
```

5. Practical Verification Guide



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



A successful lab submission should demonstrate that the student understands the sequence rather than merely copying the final script. During execution, the student should be able to point to the line that creates the SSL context, the line that opens the SMTP connection, the line that authenticates the sender, the lines that construct the message headers and body, and the line that sends the message.

For Lecture 1, the verification output should be “SMTP Login Successful” after a successful authenticated connection. For Lecture 2, the final program should report “Email sent successfully!” after `send_message()` completes. If an error occurs, the student should read the printed exception and identify whether the problem is related to credentials, addressing, ports, or network connectivity.

6. Common Beginner Mistakes

One frequent mistake is confusing the SMTP server address with a recipient email address. `smtp.gmail.com` identifies the provider’s outgoing mail server; it is not the destination inbox. Another common mistake is using the normal account password when the lab setup requires an app password. Students may also forget to use port 465 with `SMTP_SSL` in the code shown by the lecture, or they may omit the `ssl` context entirely.

A second group of errors appears in message construction. The `From` and `To` headers should contain email addresses, while `Subject` should contain a short title. The body belongs in `set_content()`, not in the `Subject` field. Finally, students sometimes establish a connection and authenticate successfully but forget that a message still needs to be constructed before `send_message(msg)` can perform the actual transmission.

7. Conceptual Recap

The entire module can be summarized as a separation between message data and message transport. `EmailMessage` represents the data: who sent the email, who should receive it, what the subject is, and what the body says. `smtpplib` represents the transport mechanism: connect to the SMTP server, authenticate, transmit the message, and close the session. `SSL/TLS` provides the secure communication layer, while the SMTP port identifies the network endpoint used by the service.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Once these fundamentals are understood, the later topics in the presentation become natural extensions rather than completely new concepts. The presentation points toward HTML email, file attachments, CC and BCC, multiple recipients, automated reports, and integration with Django as future pathways. Those topics are intentionally outside the core beginner implementation in this lab manual.

8. Practice Exercises

Exercise 1: Modify the Lecture 1 verification script so that the success message is more descriptive, for example by including the sender email address in the printed output. The goal is to reinforce that the authentication stage has completed successfully.

Exercise 2: Modify the Lecture 2 message so that the subject and body are stored in separate variables before they are assigned to `EmailMessage`. The goal is to reinforce Python variables and the difference between message metadata and message content.

Exercise 3: Run the Student Email Sender with two different recipient addresses and two different subjects. Observe how the same Python program can generate different outgoing messages without changing the core message-sending logic.

Exercise 4: Temporarily use an invalid credential in the test environment and observe the exception reported by the `try/except` block. Restore the correct credential after the observation. This exercise should be performed only in a designated testing setup.

9. Instructor Demonstration Sequence

For teaching, begin Lecture 1 with the real-world problem rather than the library name. Show the students that repeated attendance notifications, confirmations, reports, and alerts are manual communication tasks that can be triggered programmatically. Then introduce the simplified network path and define SMTP. Once students understand why a mail server is necessary, introduce `smtplib` and the SMTP server configuration.

Next, demonstrate the secure connection and authentication only. Stop there and allow students to reproduce the verification task. In Lecture 2, deliberately revisit the five-stage lifecycle and ask students



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



what is missing between authentication and sending. The answer is the message object. This creates a natural reason to introduce EmailMessage, headers, set_content(), and finally send_message(). End with the interactive Student Email Sender so that students use input(), variables, message construction, networking, authentication, and exception handling in one coherent program.

10. Source and Scope Note

This manual is based on the uploaded lecture presentation “Introduction to Email Automation & SMTP | Python Basics.” The source covers the core SMTP connection lifecycle, SMTP server role, ports 465 and 587, SSL/TLS, smtplib, authentication with app-password guidance, the Lecture 1 verification task, EmailMessage, email headers, set_content(), send_message(), the complete email automation script, basic try/except handling, common failures, the Student Email Sender mini project, and suggested future topics. The manual intentionally keeps the beginner focus of the source rather than expanding into advanced email systems.